

Constrained Delaunay tetrahedrisation refinement

Internship at the *CNR IMATI* supervised by Marco Attene and
Marco Livesu

Royer Max

ENS of Lyon

September 3, 2025

Outline

Introduction

- Motivation

- Constrained Delaunay tetrahedrisation

- State of the art

The refinement algorithm

- General principle

- Edge collapse

- A new operation

Results

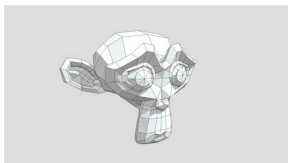
- Comparison to TetGen

Conclusion

Appendix

Big picture

Constrained Delaunay tetrahedrisation



A 3D model
(not necessarily
tetrahedrised)



The **Delaunay** tetrahedrised
model (*i.e.* a set of tetrahedra)

Applications

Physics-based simulations:

- ▶ Finite element method
- ▶ Solutions of partial differential equations
- ▶ Ray tracing



Figure: Example of finite element method

Problem

Need good quality and regularity of the mesh

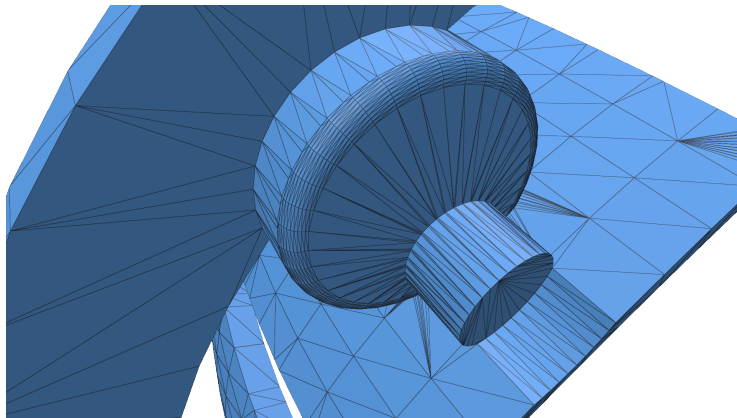


Figure: Example of a bad model

Constrained Delaunay triangulation

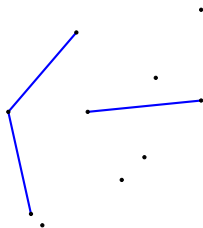


Figure: Edge constraints for Delaunay triangulation

Constrained Delaunay triangulation

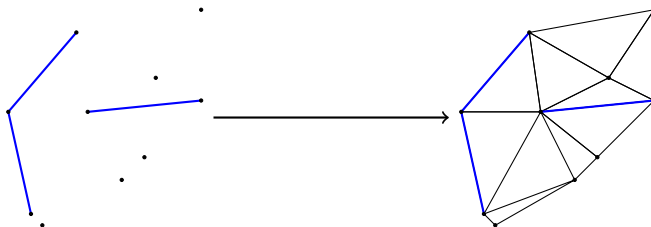


Figure: Edge constraints for Delaunay triangulation

Constrained Delaunay triangulation

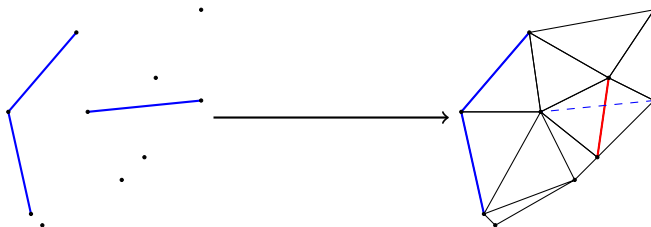


Figure: Edge constraints for Delaunay triangulation

Constrained Delaunay triangulation

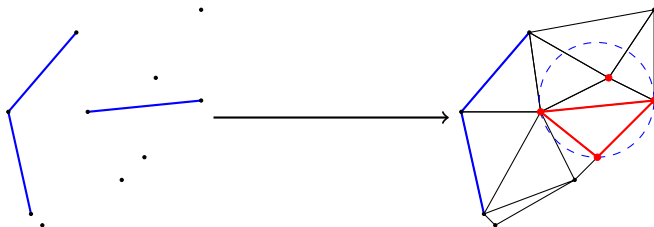


Figure: Edge constraints for Delaunay triangulation

Constrained Delaunay triangulation

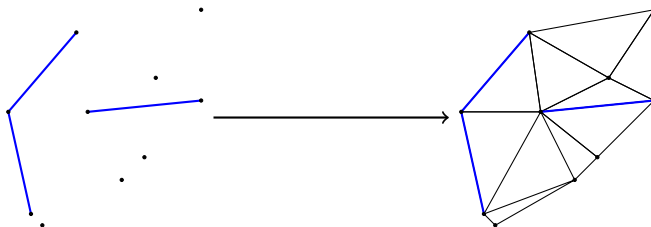


Figure: Edge constraints for Delaunay triangulation

An edge is either a constraint or is locally Delaunay

From 2D to 3D

From 2D to 3D

- ▶ Triangles become tetrahedra
- ▶ Add face constraints

From 2D to 3D

- ▶ Triangles become tetrahedra
- ▶ Add face constraints
- ▶ Slivers appear

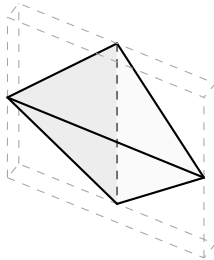


Figure: Example of sliver

From 2D to 3D

- ▶ Triangles become tetrahedra
- ▶ Add face constraints
- ▶ Slivers appear

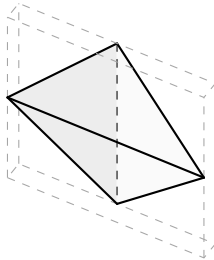


Figure: Example of sliver

- ▶ Constrained Delaunay tetrahedrisation are not always possible

TetGen [Si, 2015]

Already existing software

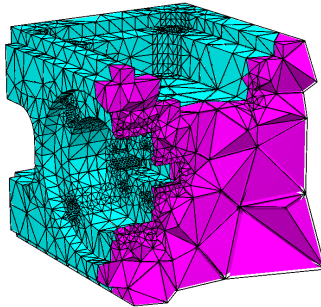


Figure: A model tetrahedrized by TetGen

- Not so robust, crashes on some models

Tetrahedral meshing in the wild (Tetwild) [Hu, 2018]

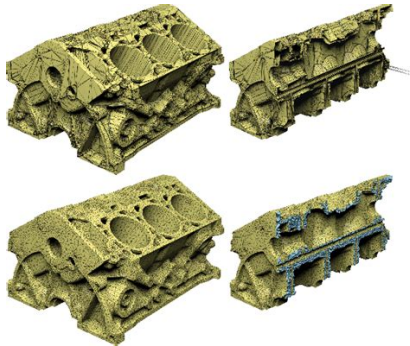


Figure: A model tetrahedrized by Tetwild

- Not strict on boundary

Quality measure

How to say if a tetrahedron is "*good*"?

Quality measure

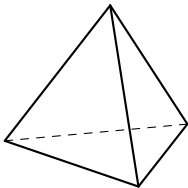
How to say if a tetrahedron is "*good*"?

- ▶ *Dirichlet* energy

Quality measure

How to say if a tetrahedron is "*good*"?

► *Dirichlet* energy

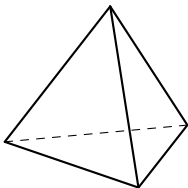


Minimized on a regular
tetrahedron (= 3)

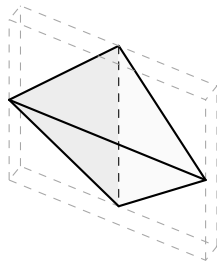
Quality measure

How to say if a tetrahedron is "good"?

► *Dirichlet* energy



Minimized on a regular
tetrahedron (= 3)



Big value on bad tetrahedron
($\sim 10^3 - 10^4$)

Local operation pass

Figure: Local operation scheme

Local operation pass

Simulation pass

Figure: Local operation scheme

Local operation pass

Simulation pass → Yields a queue Q

Figure: Local operation scheme

Local operation pass

Simulation pass $\longrightarrow$ Yields a queue Q

Applying pass, while Q not empty:

|

Figure: Local operation scheme

Local operation pass

Simulation pass → Yields a queue Q

Applying pass, while Q not empty:

Pop and apply local operation

Figure: Local operation scheme

Local operation pass

Simulation pass $\longrightarrow$ Yields a queue Q

Applying pass, while Q not empty:

Pop and apply local operation

Compute impacted elements

Figure: Local operation scheme

Local operation pass

Simulation pass $\longrightarrow$ Yields a queue Q

Applying pass, while Q not empty:

Pop and apply local operation

Compute impacted elements

Update Q

Figure: Local operation scheme

Optimization Pass

Edge split pass

Edge collapse pass

Edge swap pass

Face swap pass

Vertex smoothing pass

Tetrahedron collapse pass

Figure: Optimization process scheme

Overview

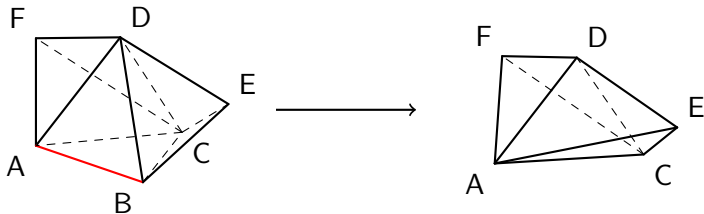


Figure: Example of collapsing an edge

Simulation

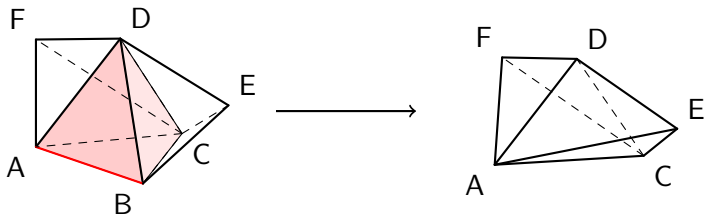


Figure: Impacted tetrahedra by an edge collapse

Simulation

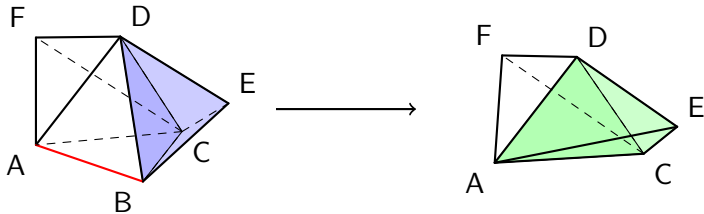


Figure: Impacted tetrahedra by an edge collapse

Simulation

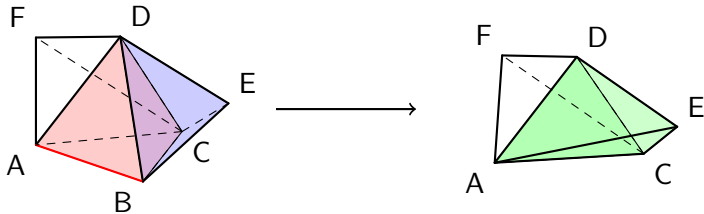


Figure: Impacted tetrahedra by an edge collapse

Quality before:

Max on **removed elements** $\cup$
unupdated elements

Quality after:

Max on **updated elements**

Topological failure

Collapsing $e := (u, v)$ can break topology

Topological failure

Collapsing $e := (u, v)$ can break topology

$$\text{Lk}(e) = \text{Lk}(u) \cap \text{Lk}(v) \quad (\text{Link condition})$$

- ▶ $\text{Lk}(u) = \text{neighbours of } u$
- ▶ $\text{Lk}(e) = \text{vertices sharing a face with } e$

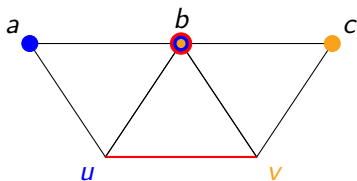


Figure: Respected link condition

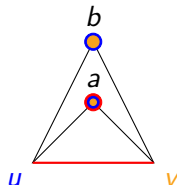


Figure: Not respected link condition

Get rid of slivers

Some slivers resisted known operation $\Rightarrow$ Create a new operation

Figure: Tetrahedron collapsing process

Get rid of slivers

Some slivers resisted known operation $\Rightarrow$ Create a new operation

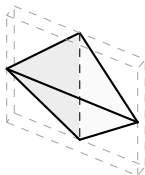


Figure: Tetrahedron collapsing process

Get rid of slivers

Some slivers resisted known operation $\Rightarrow$ Create a new operation

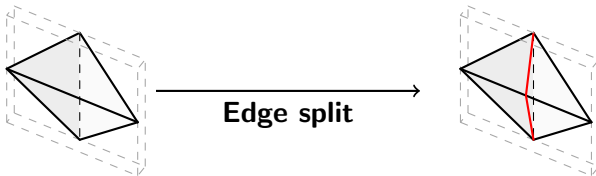


Figure: Tetrahedron collapsing process

Get rid of slivers

Some slivers resisted known operation $\Rightarrow$ Create a new operation

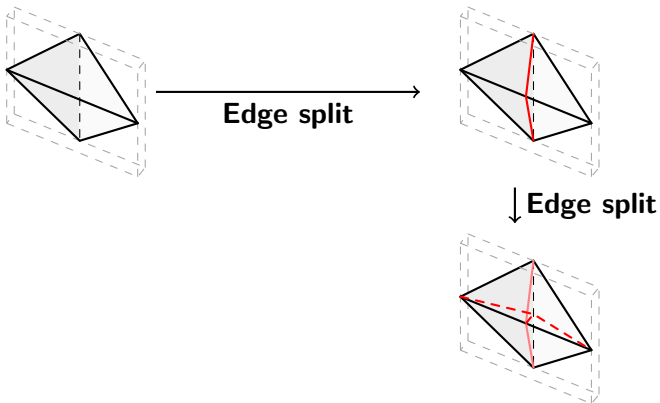


Figure: Tetrahedron collapsing process

Get rid of slivers

Some slivers resisted known operation $\Rightarrow$ Create a new operation

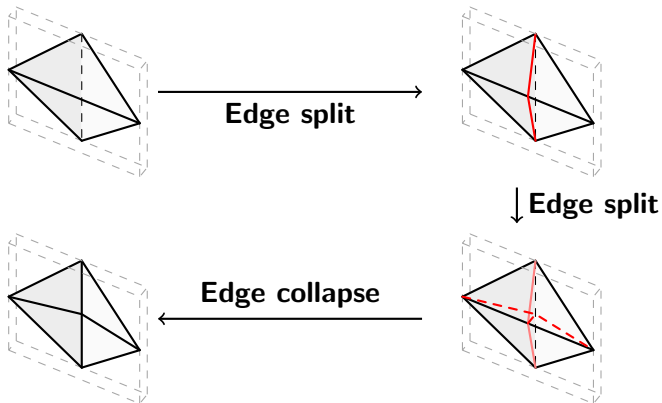


Figure: Tetrahedron collapsing process

Comparison

	CDT	TetGen
Fails	0%	25,2%

Table: Stats on the smallest 500 models of *Thingi10k*

► + More robust

Comparison

	CDT	TetGen
Fails	0%	25,2%

Table: Stats on the smallest 500 models of *Thingi10k*

- ▶ + More robust
- ▶ - Slower

	CDT	TetGen
Best on time	14,44%	85,56%

Table: Stats on the remaining models
TetGen did not failed on

Comparison

	CDT	TetGen
Fails	0%	25,2%

Table: Stats on the smallest 500 models of *Thingi10k*

- ▶ + More robust
- ▶ - Slower
- ▶ - Slightly less efficient

	CDT	TetGen
Best on time	14,44%	85,56%
Best on mean	5,35%	94,65%
Best on max	20,59%	79,41%

Table: Stats on the remaining models
TetGen did not failed on

Future work and improvements

- ▶ Optimize the speed
 - ▶ Use more specific data structures

Future work and improvements

- ▶ Optimize the speed
 - ▶ Use more specific data structures
- ▶ Create new operations
 - ▶ To remove needles

Future work and improvements

- ▶ Optimize the speed
 - ▶ Use more specific data structures
- ▶ Create new operations
 - ▶ To remove needles
- ▶ Make some operations finer
 - ▶ Split edge not always at midpoint

References



H. Si.

TetGen, a Delaunay-Based Quality Tetrahedral Mesh Generator.

ACM Trans. on Mathematical Software, 41 (2), Article 11, 2015.



Y. Hu, Q. Zhou, X. Gao, A. Jacobson, D. Zorin, D. Panozzo.
Tetrahedral Meshing in the Wild.

ACM Trans. Graph. 37, 4, Article 60, 2018.

Delaunay condition

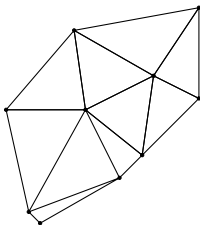


Figure: Example for Delaunay condition

Delaunay condition

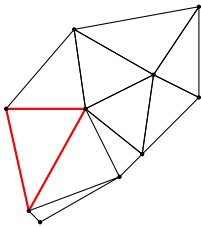


Figure: Example for Delaunay condition

Delaunay condition

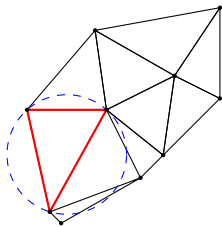


Figure: Example for Delaunay condition

Delaunay condition

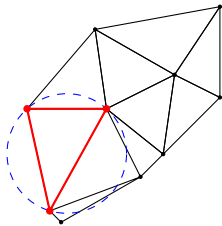
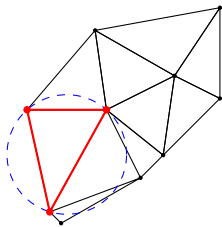


Figure: Example for Delaunay condition

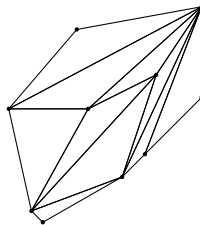
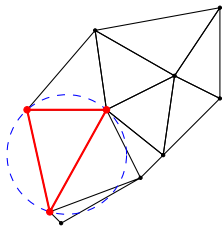
Delaunay condition



Respected

Figure: Example for Delaunay condition

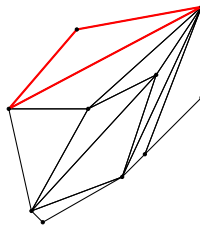
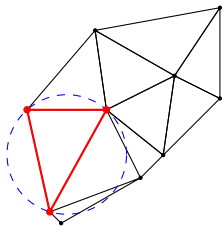
Delaunay condition



Respected

Figure: Example for Delaunay condition

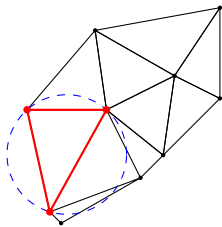
Delaunay condition



Respected

Figure: Example for Delaunay condition

Delaunay condition



Respected

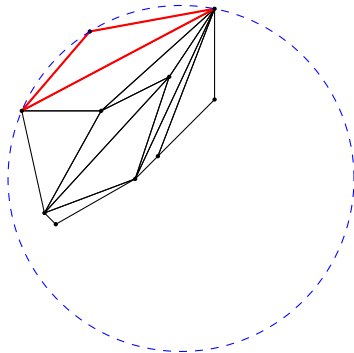
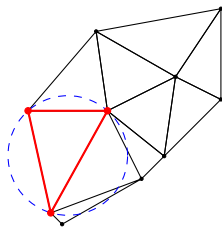


Figure: Example for Delaunay condition

Delaunay condition



Respected

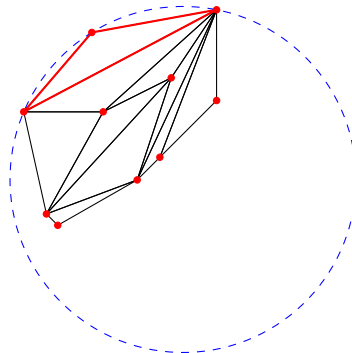
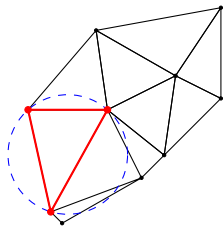
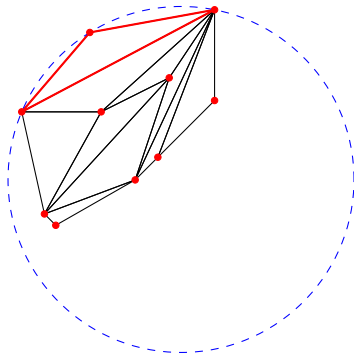


Figure: Example for Delaunay condition

Delaunay condition



Respected



Not respected

Figure: Example for Delaunay condition

An example

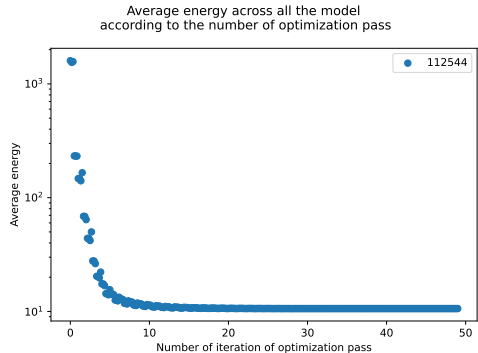
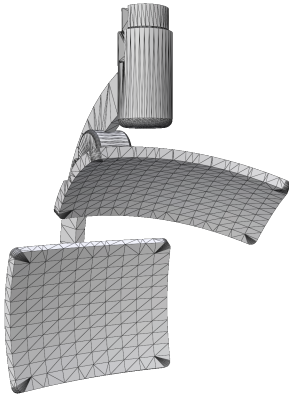


Figure: The model 112544

Evolution of the distribution

